

LAB MANUAL



**Faculty of Computing, Engineering
and The Built Environment**

COM682 - Lab 06

**Designing Storage Database and
Creating REST APIs**

Contents

1. Background	3
2. Design: Storage and API	4
2.1 Establishing a blob store	4
2.2 Establishing a Document Store (Cosmos DB)	8
2.3 Creating a REST API to consume and present images	14
2.3.1. Establish a REST endpoint to consume images from a form (IUPS)	14
2.3.1.1. Create a HTTP request trigger.....	15
2.3.1.2. Store file content in an Azure blob.....	17
2.3.1.3. Creating a Document Record in Cosmos DB.....	21
2.3.1.4. Testing the IUPS Endpoint	27
2.3.2. Establish a REST endpoint to provide a list of images stored.....	32

1. Background

In this practical exercise, students will be guided through the process of realising a simple, serverless, web solution. This solution will focus on storing and presenting images provided by end users, similar to imgur.

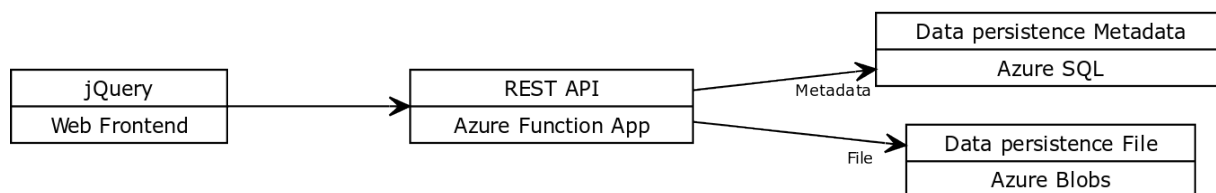
Information to be stored includes:

- A unique ID for an image record (id)
- The Filename of the record (FileName)
- A numerical representation of a user id, indicating the user that uploaded it (userID)
- A name for the user who uploaded it (userName)
- The image file (file)
- Metadata about the image file stored on Azure Blob storage (fileLocator, filePath)

The design of this solution will have 4 main components:

1. A statically generated, hosted webpage which incorporates a JavaScript library to enable a) interaction with REST services and b) manipulation of the Document Object Model (DOM).
2. An Azure Logic app to offer a REST API
3. A data storage solution, based on a relational database, using Azure SQL
4. A data storage solution, to store files, using Azure Blobs

The general architecture of this design will be as presented below:



It is suggested you follow this guide to produce your own instance of this solution.

N.B in this guide some steps are omitted from the screenshots to reduce verbosity.

2. Design: Storage and API

For this project, there are 4 main elements to establish:

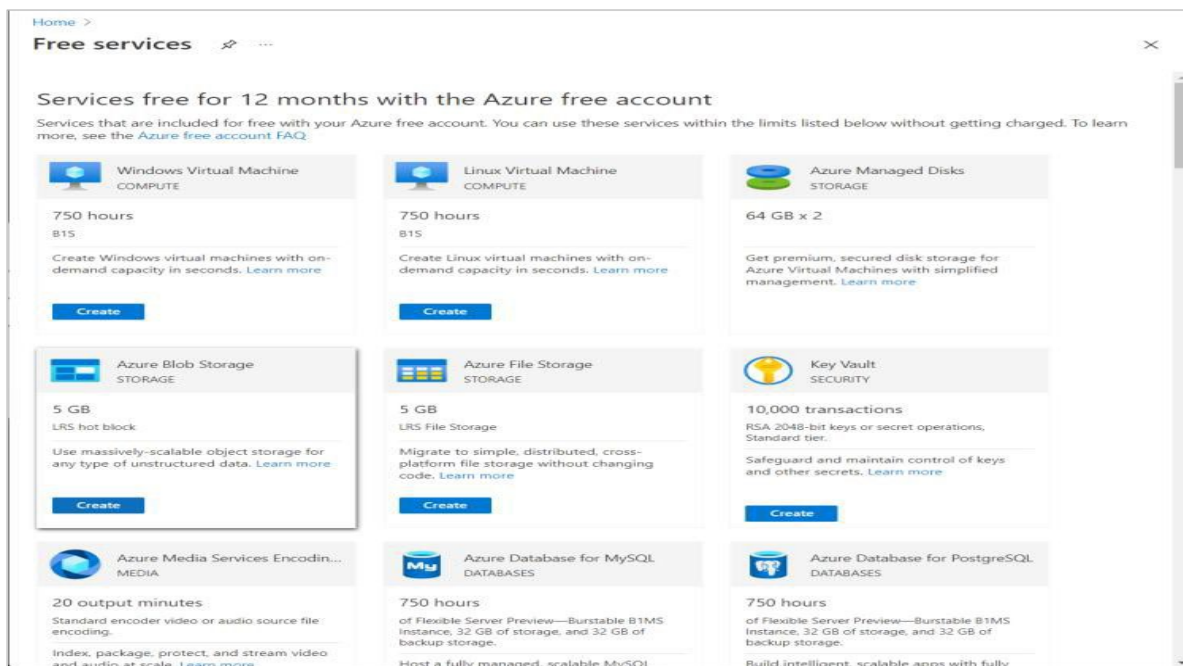
1. Blob storage to hold images
2. A document store to store metadata surrounding the images/image posts
3. The REST API to consume and present images
4. A static webpage to interact with the REST APIs and display/upload content

The design of these are explored in subsections 2.1, 2.2, 2.3 and Section 3.

2.1 Establishing a blob store

In this practical exercise, Azure blob storage will be used to hold the images for the image sharing service.

You can find this under the free services directory by searching free services in the Azure portal search bar.



Locate Azure Blob Storage and click create.

Provide a new resource group, nominate the related subscription, set a storage account name and specify the region. Use the region permitted to your account. You can find the region by looking at the Policy (Ref: Week-3 Lab). Set performance to standard.

[Home](#) > [Free services](#) >

Create a storage account ...

Basics Tags Review + create

i This is a simplified create experience specific to your free service selection. Don't see what you need? Go to the [standard create experience](#) for full customization of your storage account. ×

Azure Storage is a Microsoft-managed service providing cloud storage that is highly available, secure, durable, scalable, and redundant. Azure Storage includes Azure Blobs (objects), Azure Data Lake Storage Gen2, Azure Files, Azure Queues, and Azure Tables. The cost of your storage account depends on the usage and the options you choose below. [Learn more about Azure storage accounts](#) ↗

Project details

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription *

Resource group * [Create new](#)

Instance details

Storage account name * **i**

Region * **i**

i The combination of options below is necessary for the Azure Blob Storage free service benefit. ×

Performance * **i** **Standard:** Recommended for most scenarios (general-purpose v2 account)

Redundancy * **i** Locally-redundant storage (LRS)

[Previous](#) [Next](#) [Review + create](#)

Create a resource and click Go to Resource

[Home](#) >

 **blobstorageweek6z_1761491422378** | Overview ...

× « 🗑️ Delete ⏸️ Cancel 🔄 Redeploy ⬇️ Download 🔄 Refresh

 Overview

 Inputs

 Outputs

 Template

✓ Your deployment is complete

 Deployment name : blobstorageweek6z_1761491422378

Subscription : [Azure for Students](#)

Resource group : [Week6blob](#)

[Deployment details](#)

[Next steps](#)

[Go to resource](#)

In the Properties section, make sure the following:

- Require secure transfer for REST API operations- Enabled
- Storage account key access- Enabled

- Public network access- Enabled

Properties Monitoring Capabilities (7) Recommendations (0) Tutorials Tools + SDKs

Blob service

Hierarchical namespace	Disabled
Default access tier	Hot
Blob anonymous access	Disabled
Blob soft delete	Disabled
Container soft delete	Disabled
Versioning	Disabled
Change feed	Disabled
NFS v3	Disabled
Allow cross-tenant replication	Disabled
Storage tasks assignments	None

Security

Require secure transfer for REST API operations	Enabled
Storage account key access	Enabled
Minimum TLS version	Version 1.0
Infrastructure encryption	Disabled

Networking

Public network access	Enabled
Private endpoint connections	0
Network routing	Microsoft network routing
Endpoint type	Standard

Go to the resource and locate the containers area, this is under the data storage category on the left-hand side menu.

Home > blobstorageweek6z_1761491422378 | Overview >

blobstorageweek6z Does this storage account follow security best pra

Storage account

Search

Upload Open in Explorer Delete Move

Storage is retiring support for TLS 1.0 and 1.1 starting Nov 1, 2025. If y

Essentials

Resource group (move) : [Weeksblob](#)

Location : norwayeast

Subscription (move) : [Azure for Students](#)

Subscription ID : fa81128f-08de-4661-8799-706f7

Disk state : Available

Tags (edit) : [Add tags](#)

Properties Monitoring Capabilities (7) Recommendation

Blob service

Hierarchical namespace	Disabled
Default access tier	Hot
Blob anonymous access	Disabled
Blob soft delete	Disabled
Container soft delete	Disabled
Versioning	Disabled
Change feed	Disabled
NFS v3	Disabled
Allow cross-tenant replication	Disabled
Storage tasks assignments	None

File service

Large file share	Enabled
------------------	---------

Overview

Activity log

Tags

Diagnose and solve problems

Access Control (IAM)

Data migration

Events

Storage browser

Storage Mover

Partner solutions

Resource visualizer

▼ Data storage

Containers

File shares

Queues

Tables

> Security + networking

> Data management

> Settings

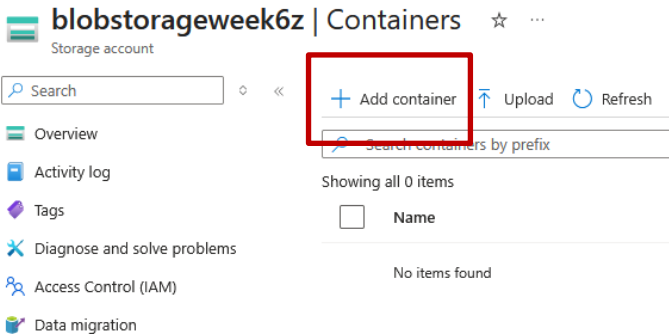
> Monitoring

> Monitoring (classic)

> Automation

An index of the current containers will be presented, by default there should only be one called **\$logs**. Click the [+ Container] option to create a new container which will be used to hold images.

[Home](#) > [blobstorageweek6z_1761491422378 | Overview](#) > [blobstorageweek6z](#)



Specify a name for the container, in this instance I provided **imagestorecom682az** ; your name should be unique. The Anonymous access level is default. Encryption scope was not configured. Create the container.

New container

Name *

imagestorecom682az

Anonymous access level ⓘ

Private (no anonymous access)

ⓘ The access level is set to private because anonymous access is disabled on this storage account.

Advanced

Encryption scope

Select from existing account scopes

☐ Use this encryption scope for all blobs in the container

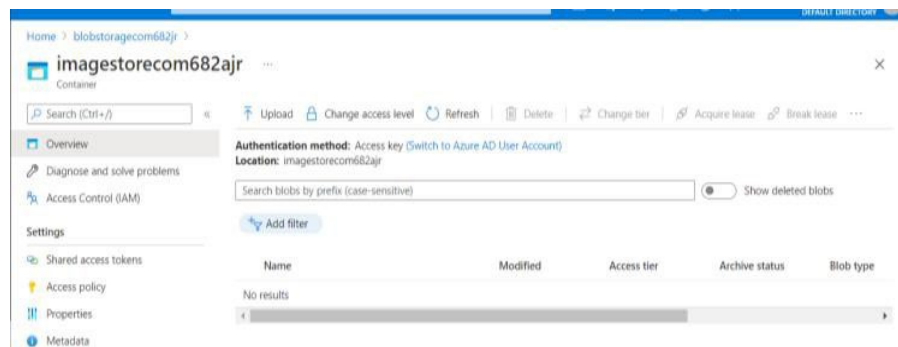
☐ Enable version-level immutability support ⓘ

ⓘ In order to enable version-level immutability support, your storage account must have versioning turned on.

Create

Give feedback

Click on new created container. You may now explore the container just created, it should be empty as there are no files present.



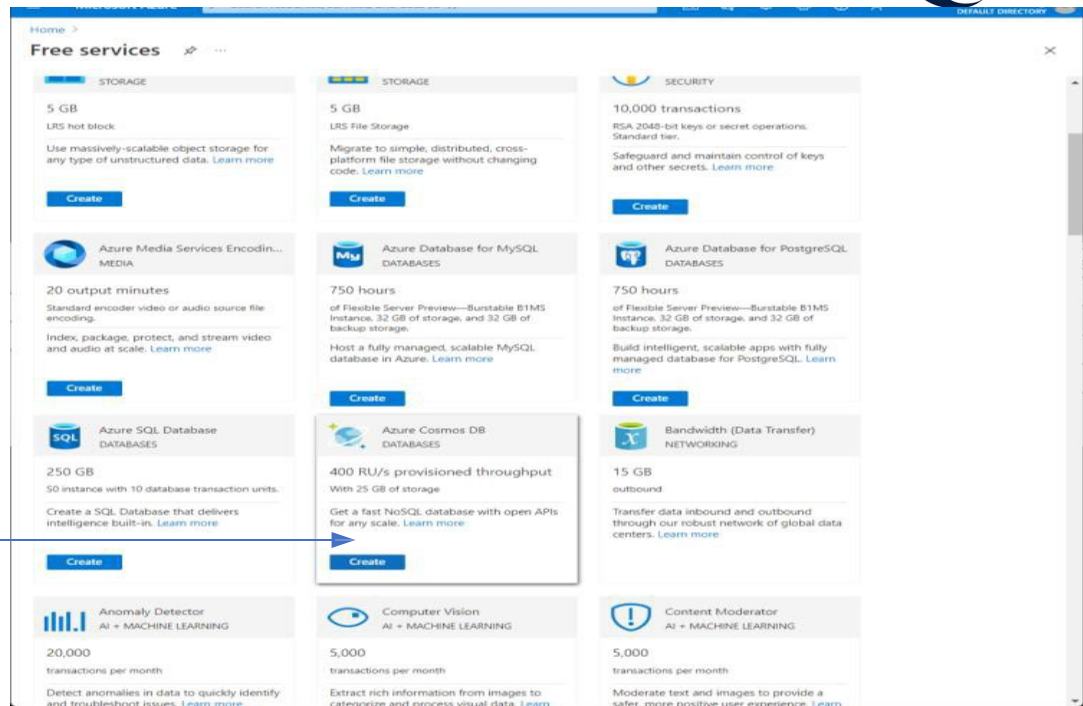
Congratulations, you have created a blob storage account and container for storage of user images.

2.2 Establishing a Document Store (Cosmos DB)

In this solution, a document store will be used to store collections of records for each image uploaded to the photo sharing system.

Cosmos DB will be used in this capacity, as previously explored in practical exercises.

In order to create a CosmosDB instance, find the Azure Cosmos DB creation option in the Azure portal. As before, this is available in the Free Services index.



Home > **Azure Cosmos DB**  [List Azure Cosmos DB accounts modified in the last 30 days using ARG.](#) [How many Azure Cosmos DB accounts do I have?](#) [Identify non-compliant Azure Cosmos DB accounts using AF](#)

Ulster University (ulster.ac.uk)

[+](#) Create [↺](#) Restore [⚙](#) Manage view [↺](#) [🔄](#) Refresh [↓](#) Export to CSV [🔗](#) Open query [🏷](#) Assign tags

[🔔](#) You are viewing a new version of Browse experience. [Click here to access the old experience.](#)

[🔍](#) Filter for any field... [Subscription equals all](#) [Type equals all](#) [Resource Group equals all](#) [Location equals all](#) [+](#) Add filter



No Azure Cosmos DB accounts to display

Create a globally distributed, multi-model, fully managed database using API of your choice. Or try it for free, up to 20k RU/s, for 30 days with unlimited renewal.

[+ Create](#)

[Try now](#)

For this solution we should select **Azure Cosmos DB for NoSQL** . The logic behind this choice was covered in previous lectures and labs.

Home > Azure Cosmos DB >

Create an Azure Cosmos DB account ...

Which API best suits your workload?

Azure Cosmos DB is a fully managed NoSQL and relational database service for building scalable, high performance applications. [Learn more](#)

To start, select the API to create a new account. The API selection cannot be changed after account creation.

Recommended APIs Others

Azure Cosmos DB for NoSQL

Azure Cosmos DB's core, or native API for working with documents. Supports fast, flexible development with familiar SQL query language and client libraries for .NET, JavaScript, Python, and Java.

[Create](#) [Learn more](#)

Azure Cosmos DB for MongoDB

Fully managed database service for apps written for MongoDB. Recommended if you have existing MongoDB workloads that you plan to migrate to Azure Cosmos DB.

[Create](#) [Learn more](#)

Next, specify a new resource group and account name for the Cosmos DB instance.

Set the location and capacity mode as appropriate for your needs. As the below was created with a free tier only account, provisioned throughput was selected.

On accounts with credit, such as the UU accounts, it may be best to select serverless. Use your judgement as informed by previous practical's and lecture materials.

Workload type: Learning

Availability Zones: Enable (and select your provisioned region)

Capacity mode: Provisioned/ Serverless (both should work)

[Home](#) > [Azure Cosmos DB](#) > [Create an Azure Cosmos DB account](#) >

Create Azure Cosmos DB Account - Azure Cosmos DB for NoSQL

Project Details

Choose a workload type that best aligns with your goals. This helps us provide an optimized starting point for your Azure Cosmos DB account setup. Don't worry—no matter which workload type you choose, you can cu

Workload Type *

Learning

Best for beginners. Low cost, easy setup, and quick exploration to learn Azure Cosmos DB.

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Azure for Students

Resource Group *

(New) Week6cosmosdb

[Create new](#)

Instance Details

Account Name *

week6cosmosdbz

Configure availability zone settings for your account. You cannot change these settings once the account is created.

Availability Zones

☐ Enable ☒ Disable

Location *

(US) East US 2 EUAP

Available locations are determined by your subscription's access and availability zone support (if that is enabled). If you don't see or cannot select your desired location, please [Click here for more details on how to create a region access request](#)

Unsure of your traffic patterns? Explore Azure Cosmos DB's serverless offering if you're just starting out and skip capacity planning by only paying for the request units (RUs) you consume. [Learn more](#)

Capacity mode

☒ Provisioned throughput

Request units per second (RU/s) are pre-configured and billed hourly, offering guaranteed throughput with two modes—**Autoscale**, which dynamically adjusts within a 1-

☐ Serverless

Ideal for intermittent or unpredictable traffic. Billed only for consumed RU/s, scaling on-demand without capacity planning or minimum charges.

[Learn more about capacity mode](#)

With Azure Cosmos DB free tier, you will get the first 1000 RU/s and 25 GB of storage for free in an account. You can enable free tier on up to one account per subscription. Estimated \$64/month discount per account.

Apply Free Tier Discount

☒ Apply ☐ Do Not Apply

[Review + create](#)

[Previous](#)

[Next: Global distribution](#)

Click next and then configure Global Distribution as appropriate

[Basics](#)
[Global Distribution](#)
[Networking](#)
[Backup Policy](#)
[Encryption](#)
[Tags](#)
[Review + create](#)

Global Distribution

Configure global distribution and regional settings for your account. You can also change these settings after the account is created.

Geo-Redundancy

☐ Enable
 ☒ Disable

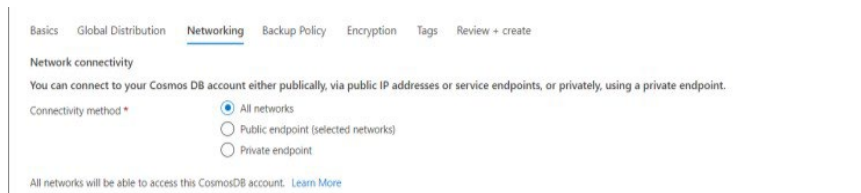
Multi-region Writes

☐ Enable
 ☒ Disable

Availability Zones

☐ Enable
 ☒ Disable

Click next to configure networking and set the connectivity to All Networks



Basics Global Distribution **Networking** Backup Policy Encryption Tags Review + create

Network connectivity

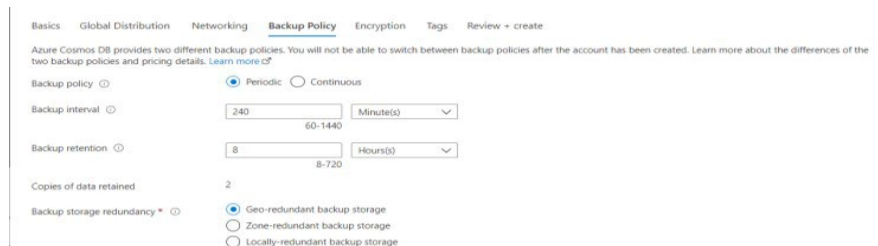
You can connect to your Cosmos DB account either publicly, via public IP addresses or service endpoints, or privately, using a private endpoint.

Connectivity method *

- ☒ All networks
- ☐ Public endpoint (selected networks)
- ☐ Private endpoint

All networks will be able to access this CosmosDB account. [Learn More](#)

Click next to set the Backup Policy. In most cases, default options should be sufficient



Basics Global Distribution Networking **Backup Policy** Encryption Tags Review + create

Azure Cosmos DB provides two different backup policies. You will not be able to switch between backup policies after the account has been created. [Learn more about the differences of the two backup policies and pricing details.](#)

Backup policy ☒ Periodic ☐ Continuous

Backup interval Minute(s)

Backup retention Hours(s)

Copies of data retained

Backup storage redundancy * ☒ Geo-redundant backup storage ☐ Zone-redundant backup storage ☐ Locally-redundant backup storage

Click next to specify encryption options, Service-managed key should be sufficient for most use cases.

[HOME](#) > [AZURE COSMOS DB](#) > [Create an AZURE COSMOS DB ACCOUNT](#) >

Create Azure Cosmos DB Account - Azure Cosmos DB for NoSQL

Basics Global distribution Networking Backup Policy **Security** Tags Review + create

Key-based Authentication

Key-based authentication uses account keys that are granted permissions to the entire Cosmos DB account. Disabling these me

Key-based Authentication ☒ Enable ☐ Disable

Data Encryption

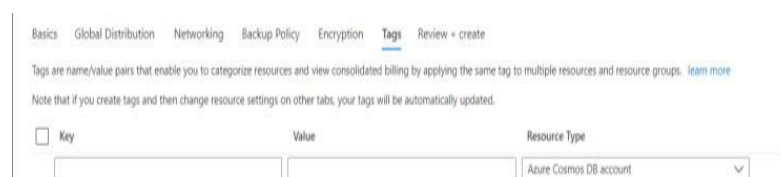
Azure Cosmos DB encryption protects your data at rest by seamlessly encrypting your data as it's written in our datacenters, an

By default your Azure Cosmos DB account is encrypted at rest using service-managed keys. At the moment, you will not be able

Data Encryption *

- ☒ Service-managed key
- ☐ Customer-managed key (CMK)

Click next, then specify tags as appropriate.



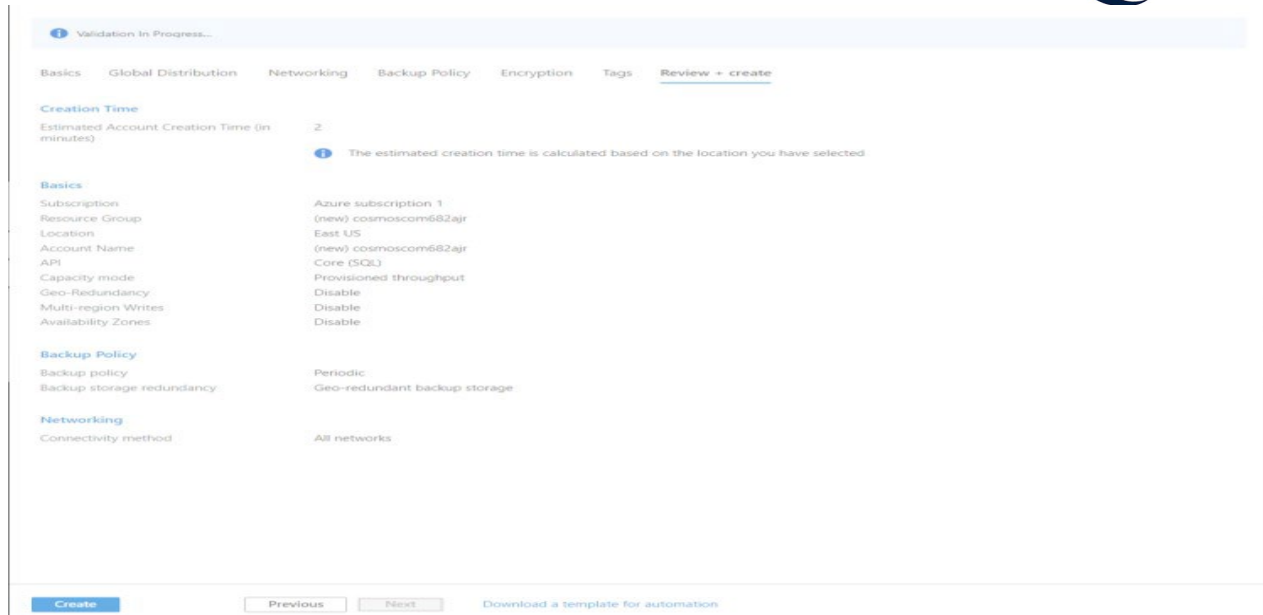
Basics Global Distribution Networking Backup Policy Encryption **Tags** Review + create

Tags are name/value pairs that enable you to categorize resources and view consolidated billing by applying the same tag to multiple resources and resource groups. [learn more](#)

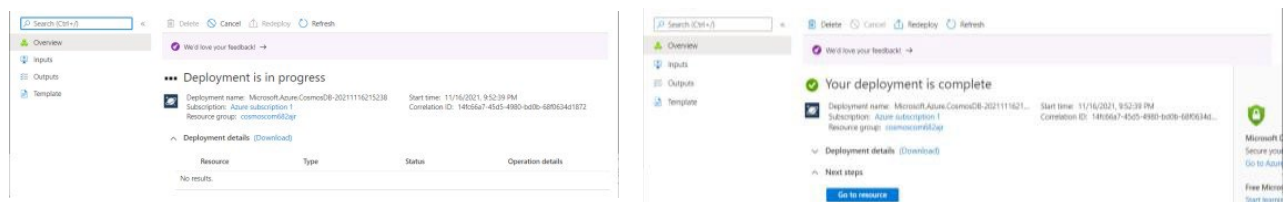
Note that if you create tags and then change resource settings on other tabs, your tags will be automatically updated.

Key	Value	Resource Type
		Azure Cosmos DB account

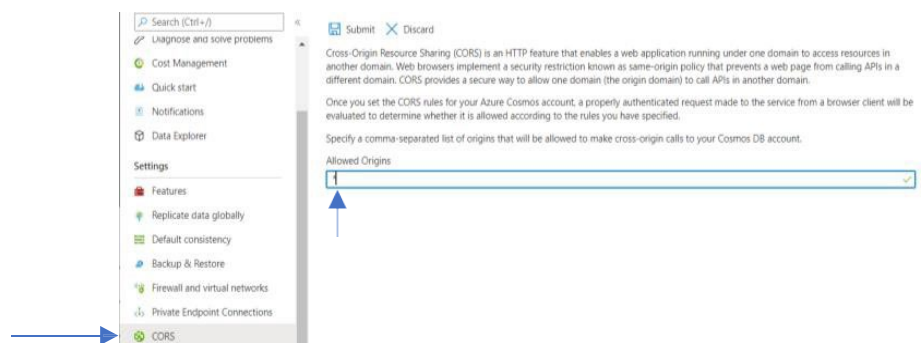
Click next, then you configuration will be validated.



Upon successful validation, the Create button will be enabled. You may then click create to start deployment.

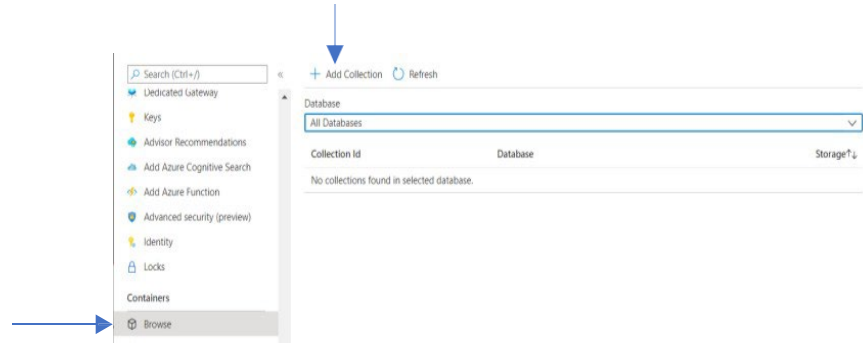


At the resource, click on Settings, navigate to the CORS option and allow all origins by specifying *. Now click submit.



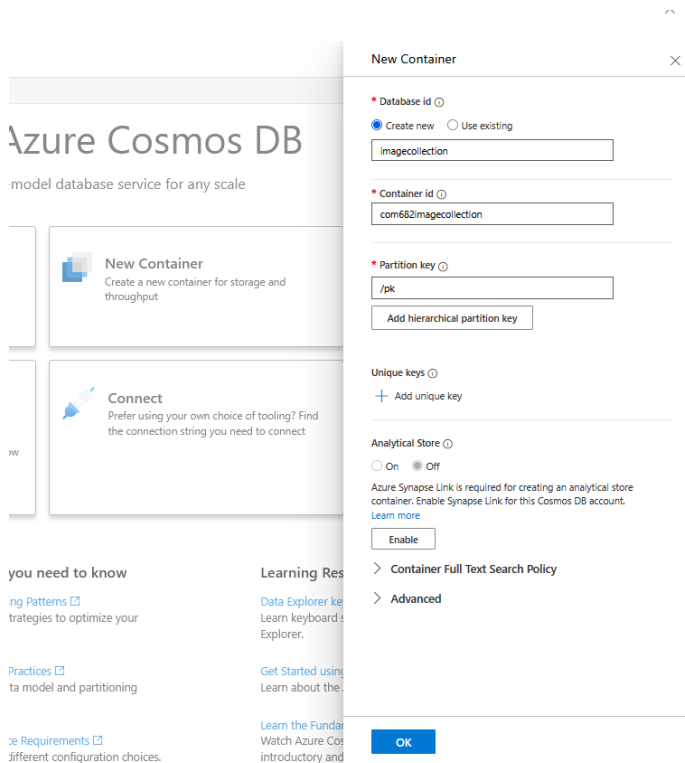
Next, navigate to Browse in the containers option of the left-hand menu.

Following that, click **[+ Add Collection]** to specify a collection to host the image related posts.

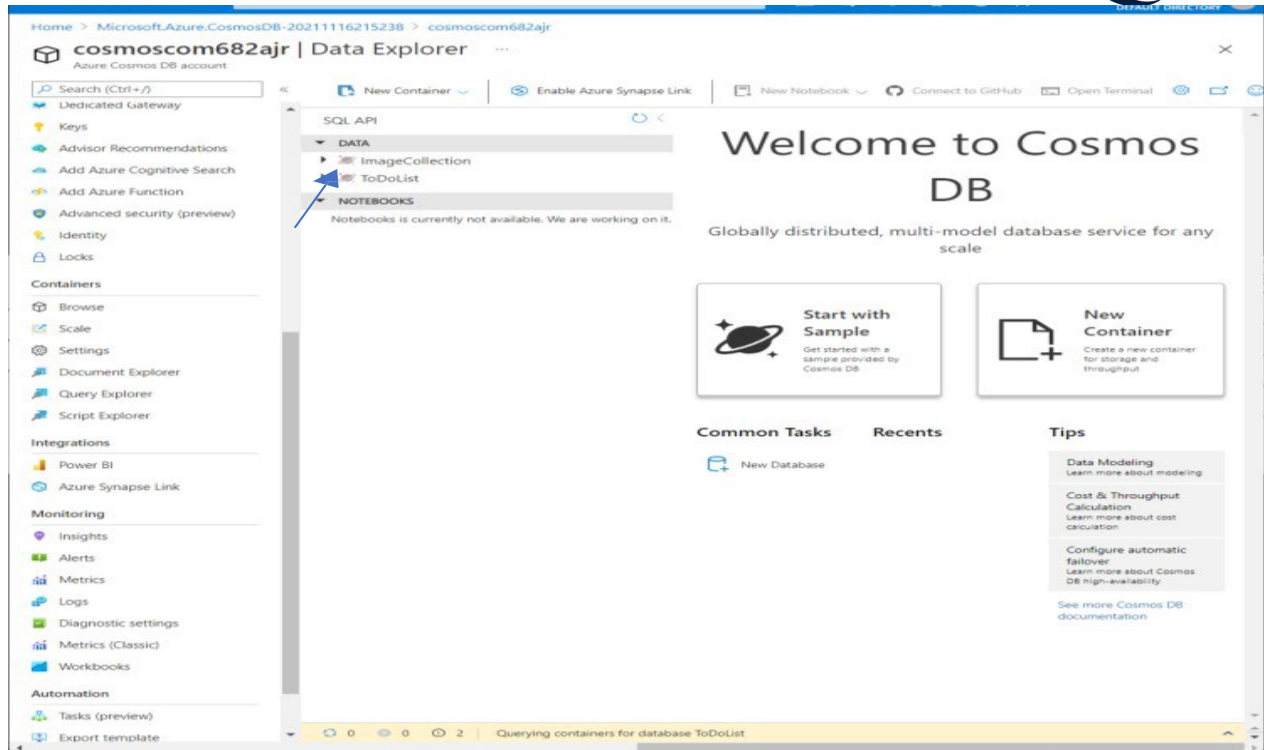


Click on + New Container

Specify a name for the database, and specify a container ID, partition key = /pk.



Click Ok, then the collection will be created. It will then appear in the Data menu.



Congratulations, a document store for the application has been created. This store will hold metadata about each post, including a link to the Azure Blob representing the image.

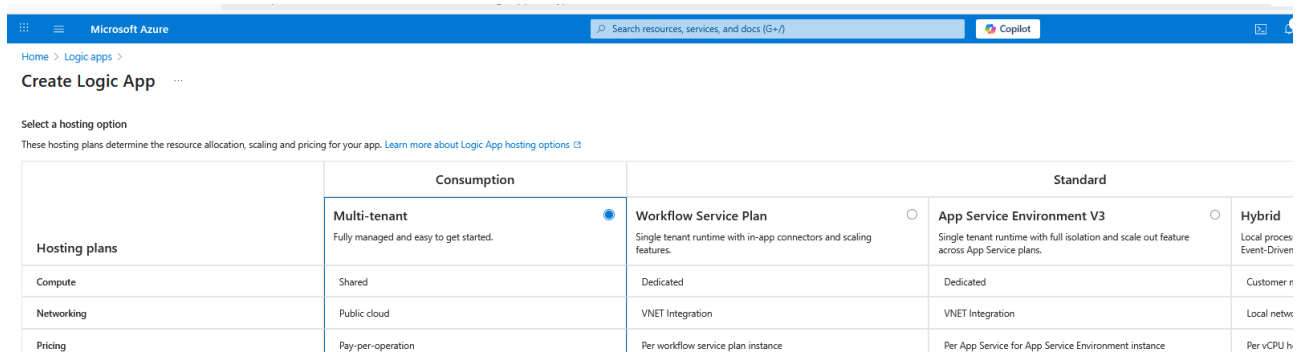
2.3 Creating a REST API to consume and present images

Next, create a logic app to serve as the rest endpoint host. This should be created from the Free services index or as previously described in multiple previous practical exercises.

Once you have produced a logic app, you can then define the workflows.

2.3.1. Establish a REST endpoint to consume images

We need to create an endpoint that will consume images and metadata and store this on CosmosDB and the Azure blob storage. This will consume POST requests from clients.



	Consumption	Standard		
Hosting plans	Multi-tenant Fully managed and easy to get started.	☐ Workflow Service Plan Single tenant runtime with in-app connectors and scaling features.	☐ App Service Environment V3 Single tenant runtime with full isolation and scale out feature across App Service plans.	☐ Hybrid Local process Event-Driven
Compute	Shared	Dedicated	Dedicated	Customer managed
Networking	Public cloud	VNET Integration	VNET Integration	Local network
Pricing	Pay-per-operation	Per workflow service plan instance	Per App Service for App Service Environment instance	Per vCPU hour

In the below screenshot we have created a logic app called **com683imageajr**.

The process for creating logic apps has been covered by previous practical exercises, ensure you use the Free F1 tier. If there are issues deploying to the free tier, use the consumption model as shown below.

Home > Logic apps > Create Logic App >

Create Logic App (Multi-tenant) ...

Basics Tags Review + create

Create a logic app, which lets you group workflows as a logical unit for easier management, deployment and sharing of resources. Workflows let you connect your business-critical apps and services with Azure Logic Apps, automating your workflows without writing a single line of code.

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Resource Group * [Create new](#)

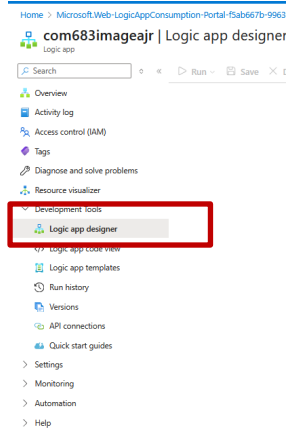
Instance Details

Logic App name *

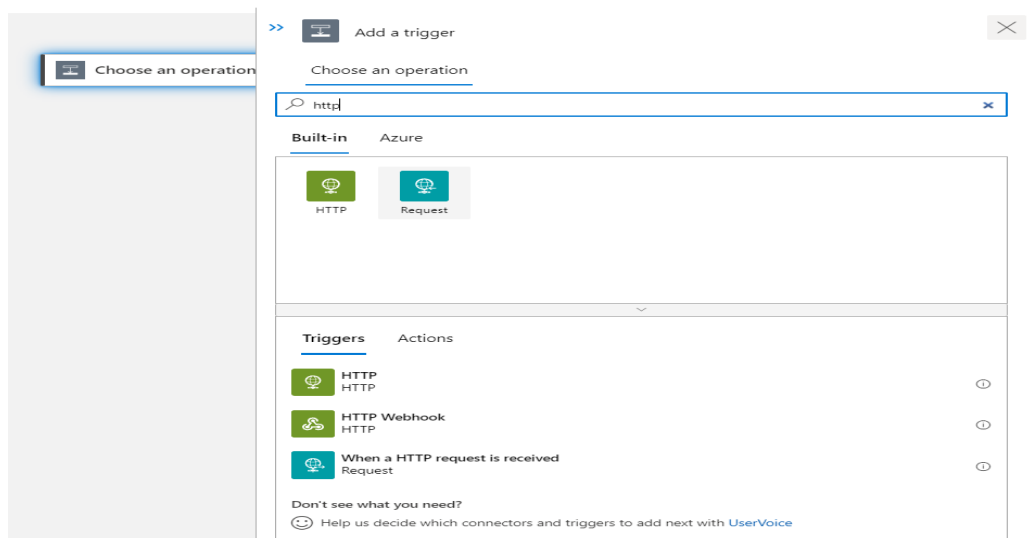
Region *

Enable log analytics * ☐ Yes ☒ No

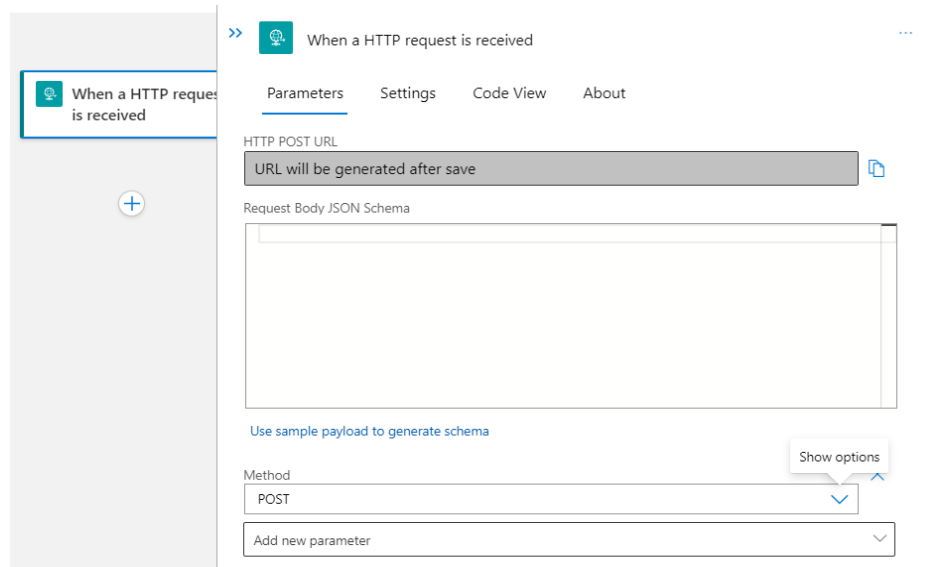
2.3.1.1. Create a HTTP request trigger



Next create a new Request trigger.

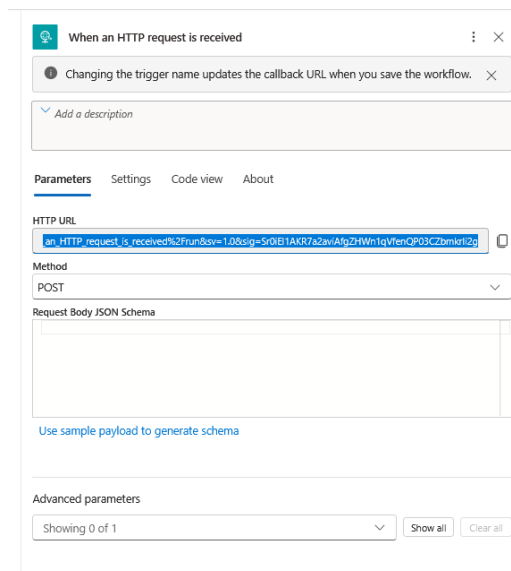


Set the method to POST.



The screenshot shows the configuration for a Logic App trigger named "When a HTTP request is received". The interface includes tabs for "Parameters", "Settings", "Code View", and "About". The "Parameters" tab is active, showing the "HTTP POST URL" field with the placeholder text "URL will be generated after save". Below this is the "Request Body JSON Schema" field, which is currently empty. A link "Use sample payload to generate schema" is provided. The "Method" dropdown is set to "POST". At the bottom, there is an "Add new parameter" button.

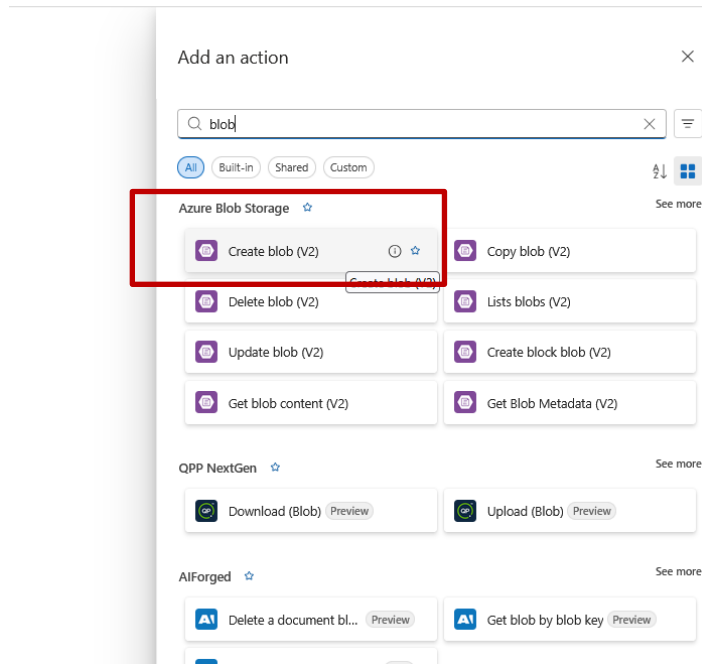
After the logic app is saved the URI will be generated, take a note of this as it will be our IUPS endpoint (**I**mage **U**pload **S**ervice).



The screenshot shows the same Logic App trigger configuration interface after it has been saved. The "HTTP URL" field now contains a generated URI: "an_HTTP_request_is_received%2funsy=1.0&u-g=50B1AKR7aZavA5p2tWn1qYenQP03CZbmrl2a". The "Method" dropdown remains set to "POST". The "Request Body JSON Schema" field is still empty. At the bottom, there is an "Advanced parameters" section with a dropdown set to "Showing 0 of 1" and buttons for "Show all" and "Clear all".

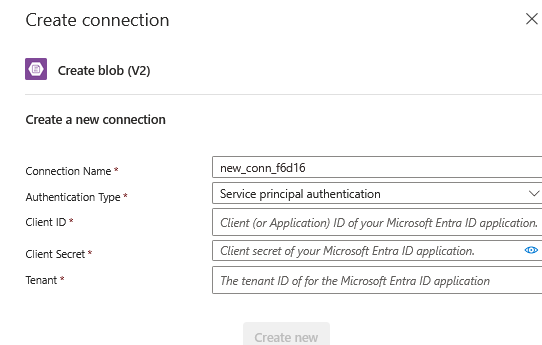
2.3.1.2. Store file content in an Azure blob

Next, create a new operation to create a blob. This is under the Azure options and called

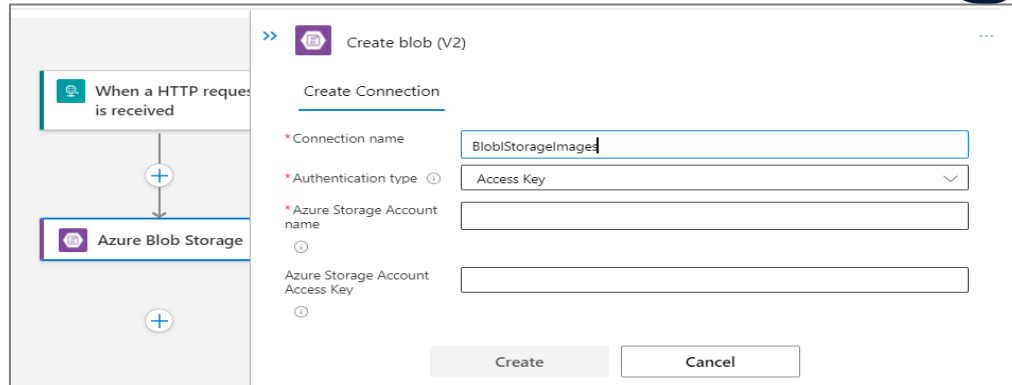


Create Blob (v2). This will be used to create an image from an uploaded form.

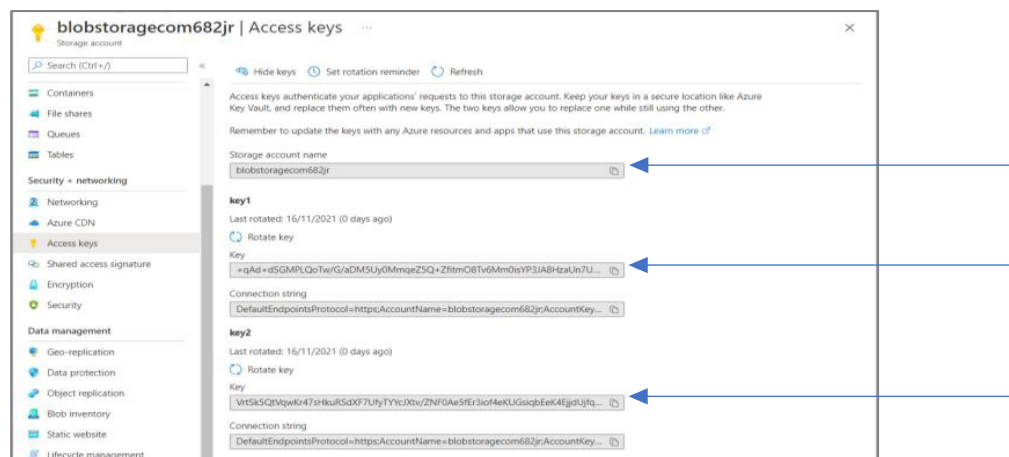
Next, create a connection to be used by the Create Blob (v2) method.



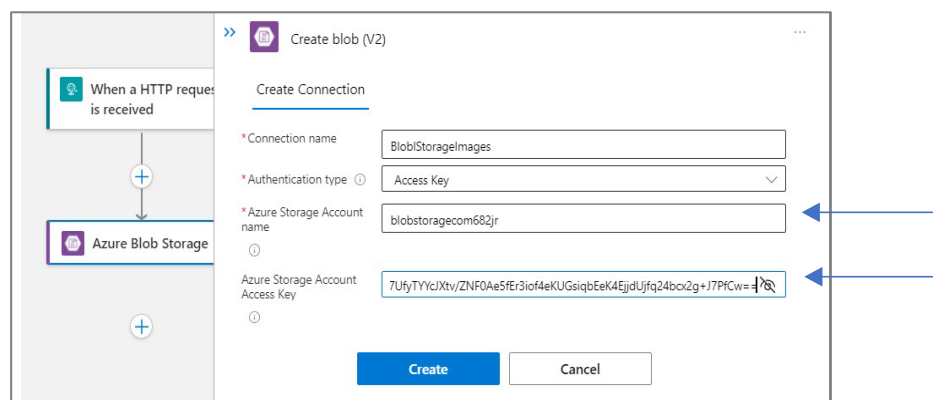
set Authentication type to Access Key. Next you will need to get the storage account and access key parameters.



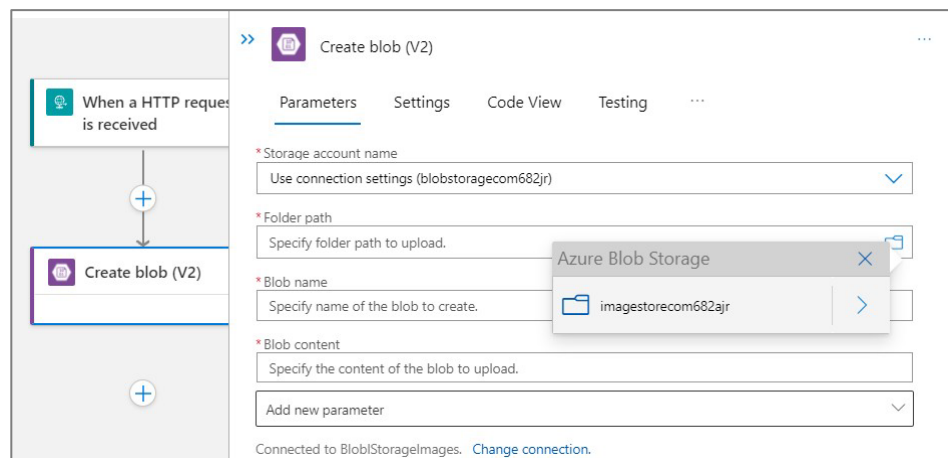
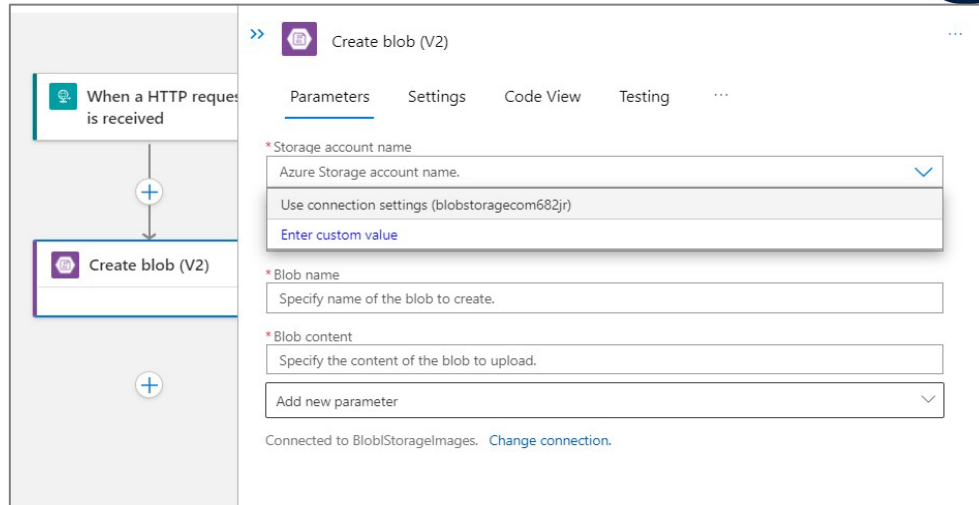
These details can be obtained from the access keys section of the **blob storage** which was previously set up in 2.1.



Note the account storage name and an access key, then provide these to the connection parameters in the Create Connection interface.



Next select the appropriate storage account name and folder. This will be as previously configured.



Next, set the blob name as `ticks(utcNow())`. This sets the blob file name to a variation of UNIX time, ensuring uniqueness.

As this API is intended to consume uploaded images from web clients we will assume content will be provided using POST with data modelled in the **multipart/form-data** style. This impacts how we may access the data provided.

Specifically, we will need to use the `triggerFormDataValue({x})` routine. In this routine we replace `{x}` with the key name of a key value pair which the form data is modelled in.

For example, in the below screenshot we are accessing the file data uploaded.

Create blob (V2)

Parameters Settings Code view Testing About

Storage account name or blob endpoint *

Use connection settings(blobstorageweek6z)

Folder path *

/imagestorecom682az

Blob name *

Specify name of the blob to create.

'Blob name' is required.

Insert expression (you can also add by typing '/' in the editor)

Info

ticks(utcNow())

Function Dynamic content

tick

Date and time functions

ticks(timestamp)

Returns the number of ticks (100 nanoseconds interval) since 1 Januar...

Add

Create blob (V2)

Parameters Settings Code view Testing About

Storage account name or blob endpoint *

Use connection settings(blobstorageweek6z)

Folder path *

/imagestorecom682az

Blob name *

Specify name of the blob to create.

Blob content *

Specify the content of the blob to upload.

Advanced parameters

Showing 0 of 1

Show all Clear all

Connected to new_conn_f6d16. Change connection

Next, set the blob content to *triggerFormDataRowValue('File')*, this will correspond to an item called file on the form which uploads to this endpoint.

When a HTTP request is received

Create blob (V2)

Storage account name

Use connection settings (blobstoragecom682jr)

Folder path

/imagestorecom682ajr

Blob name

ticks(...)

Blob content

triggerFormDataRowValue(...)

Add dynamic content

Dynamic content Expression

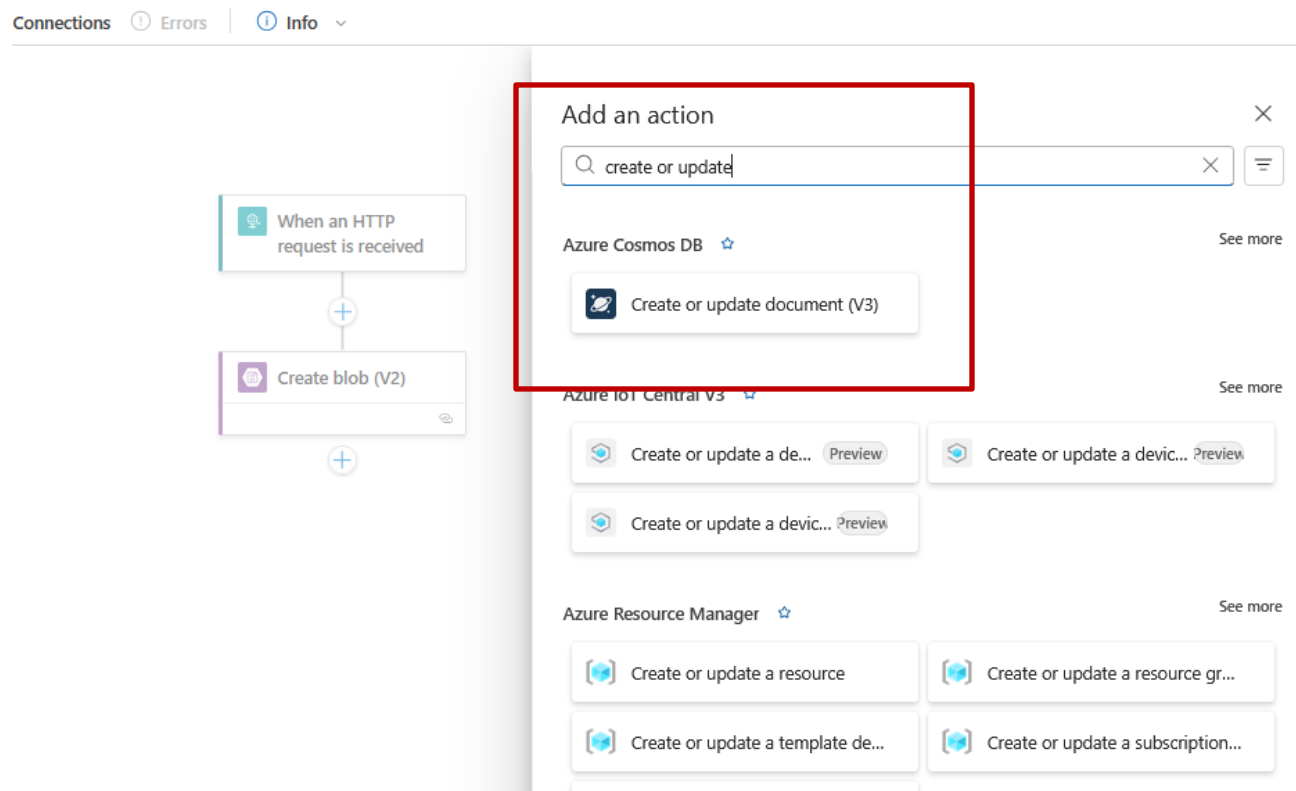
triggerFormDataRowValue('File')

Update

At this stage, the workflow will take a multi-part form post and extract a file and store it in the Azure Blob storage. Next, there is a need to store the contents of this image upload to a document collection.

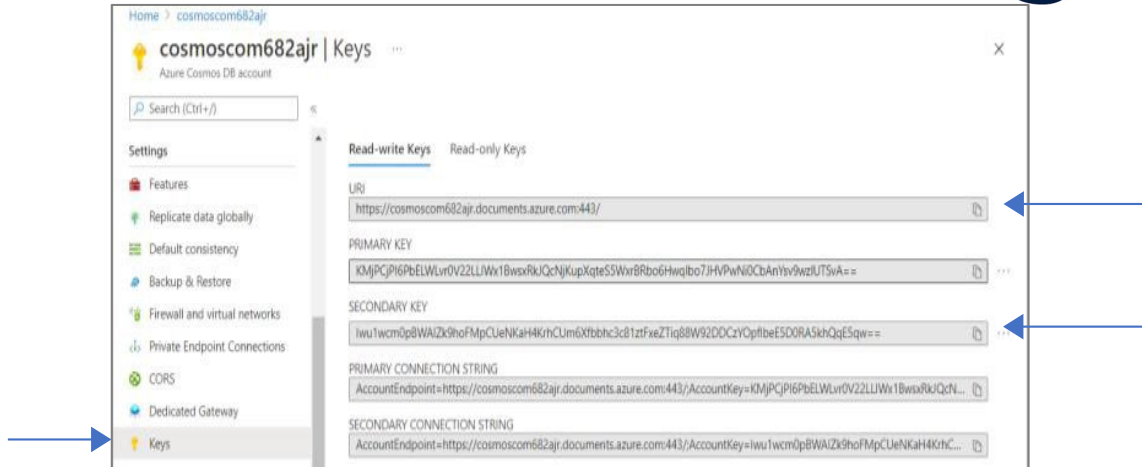
2.3.1.3. *Creating a Document Record in Cosmos DB*

To achieve this, add a stage to the workflow which will create a document in Cosmos DB. This should be under the Azure routine **Create or update document (v3)**

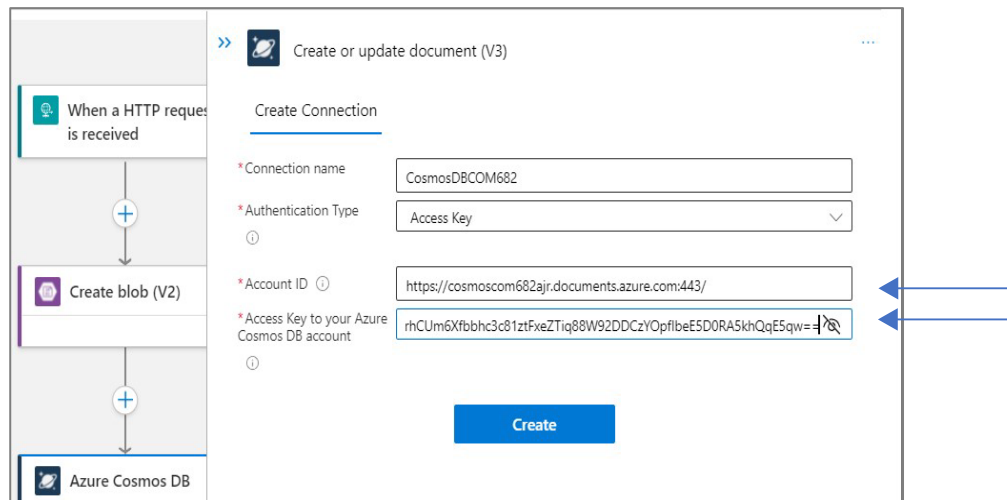


In the next stage we will need the access credentials for the related cosmos database. In order to do this, open a new browser/tab to the overview of the database and access the keys section. Navigate to the read/write key section as this is appropriate for our needs.

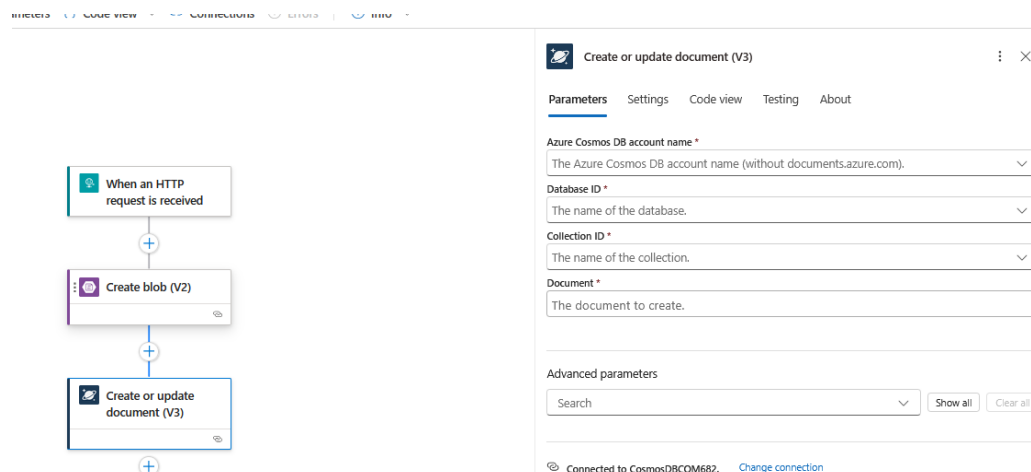
The important elements are the URI/Account ID and the primary and secondary keys. Note these.



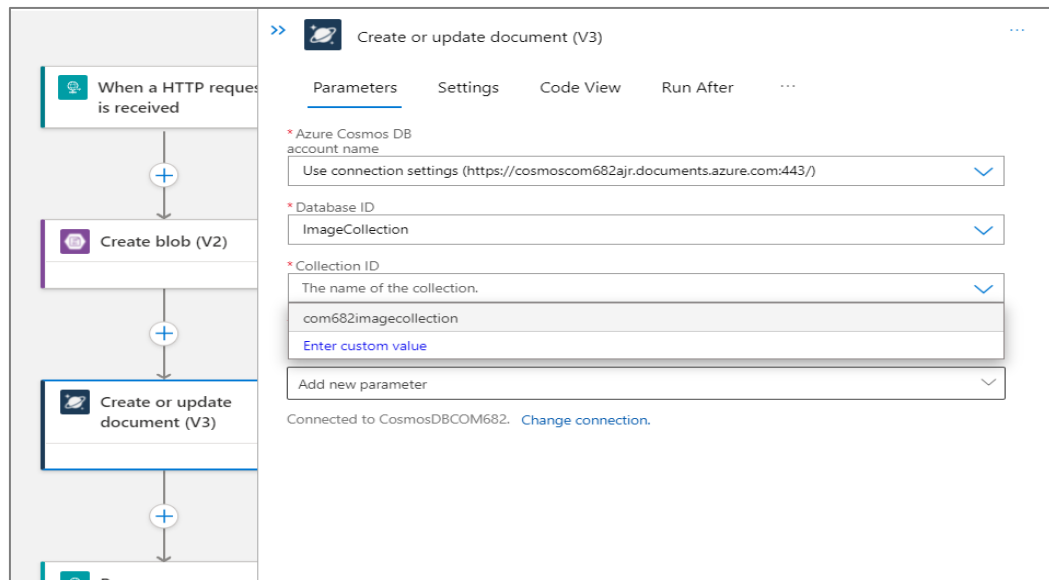
Copy these to the relevant areas in the designer for the logic app. Account ID is referred to as in the Keys overview panel URI. The access key can be either the secondary or primary keys.



On creation of the connection, then you will be able to select the parameters for the Cosmos DB to be used. In this case we select the newly created connection string.



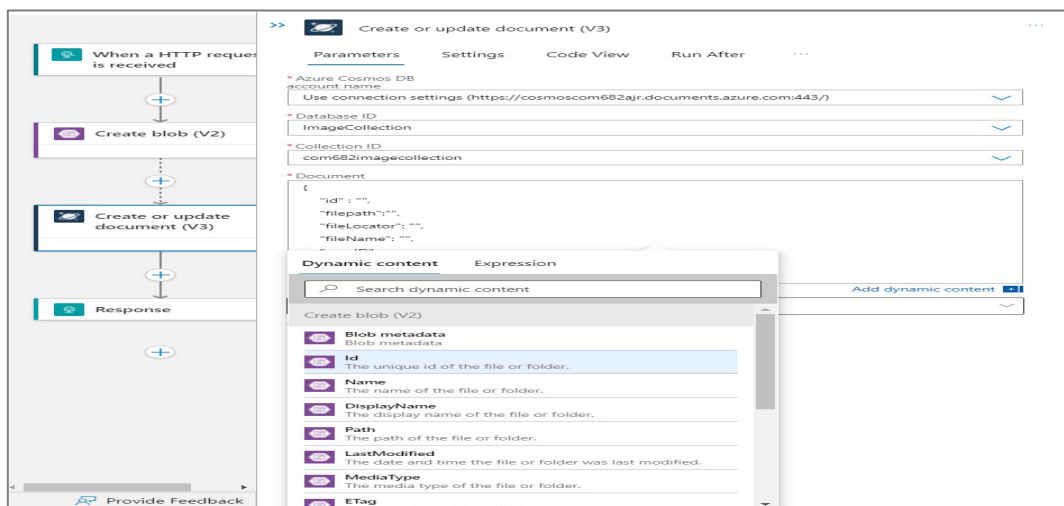
Following this, the appropriate Database ID is specified as is the collection ID. These were determined in the setup of Cosmos.



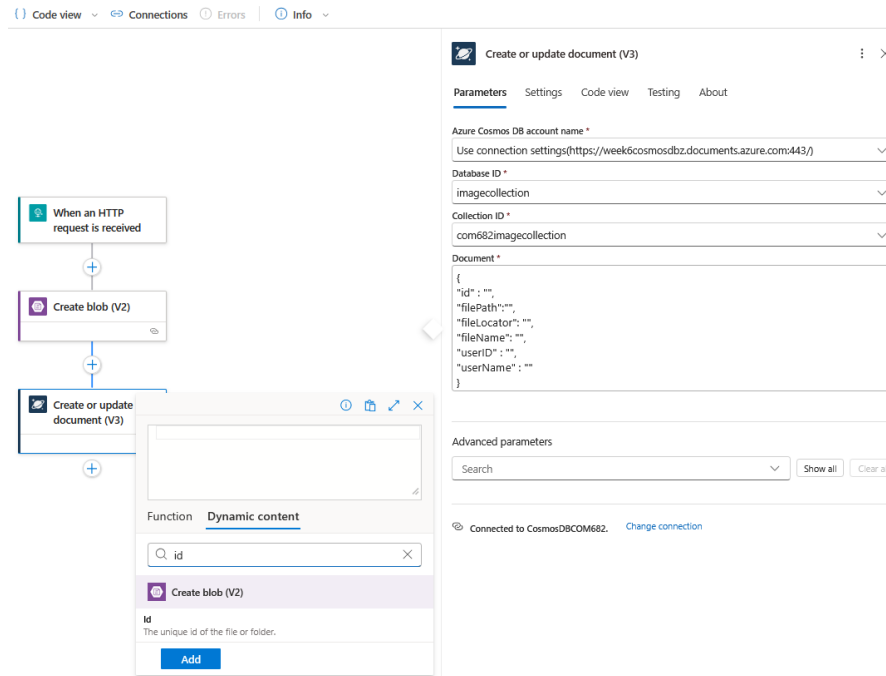
Next, specify a JSON document to insert into the collection. The template for this will follow this outline:

```
{
  "id" : "",
  "filePath": "",
  "fileLocator": "",
  "fileName": "",
  "userID" : "",
  "userName" : ""
}
```

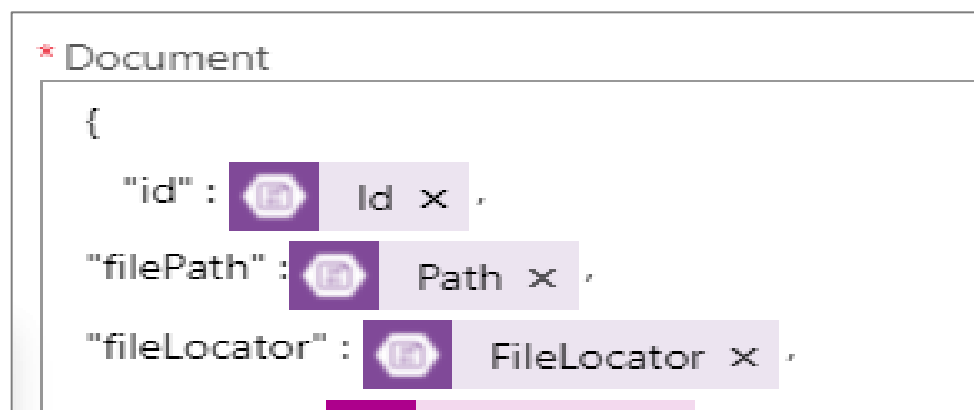
It is important to modify the content of the empty “” literals (JSON values) with parameters which will be provided by clients consuming this API.



3 of these literals will be replaced with dynamic content from the blob operation. Replace the empty literals for id with the ID blob dynamic object, which looks like:

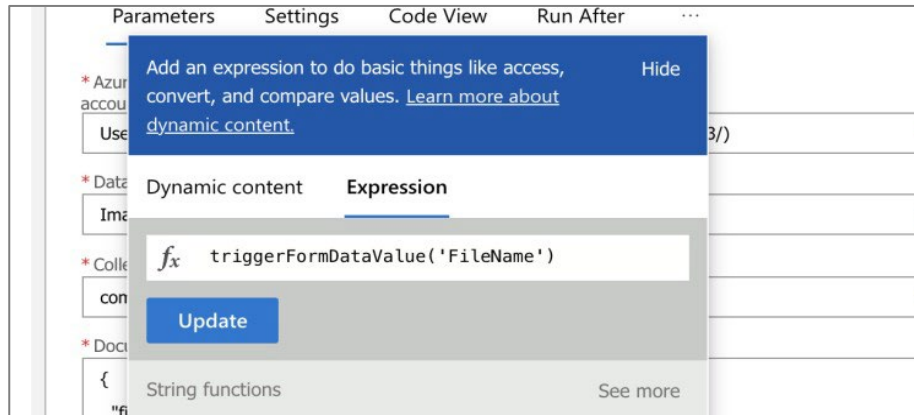


Next change the **filePath** empty literals to the **Path** content from the blob object and then the same with the **fileLocator** Literal and blob object **FileLocator**. The document will resemble the below.

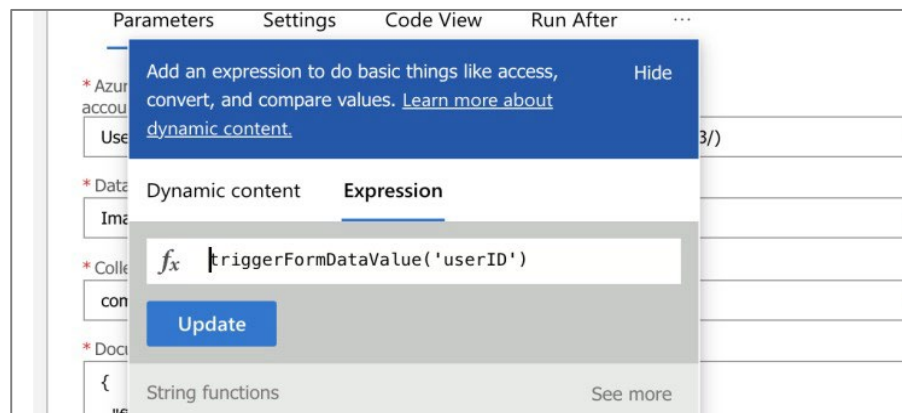


Next, you must provide expressions for the remaining literals, these will get data from the multipart form opposed to the blob object. Therefore, we need to use the `triggerFormDataValue({x})` routine again.

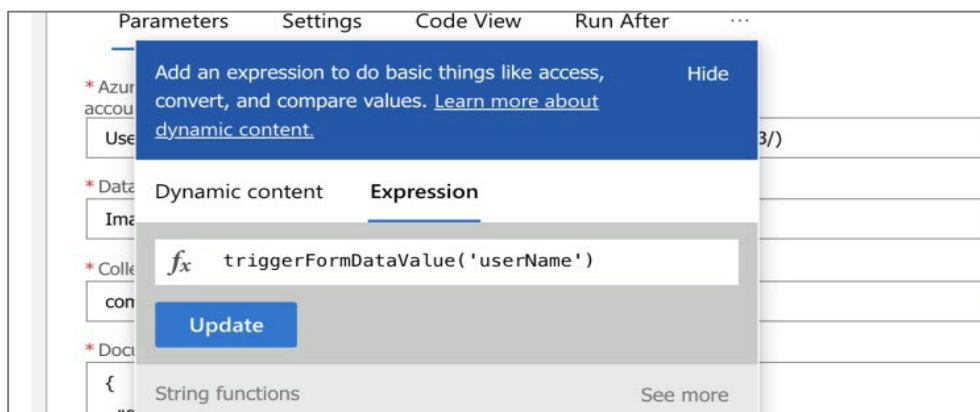
For the fileName empty literal use the expression `triggerFormDataValue('FileName')` .



For the userID empty literal use the expression `triggerFormDataValue('userID')` .



For the userName empty literal use the expression `triggerFormDataValue('userName')`.



Ultimately, the completed document will appear as below:


Create or update document (V3)

⋮

×

Parameters
Settings
Code view
Testing
About

Azure Cosmos DB account name *

Use connection settings(https://week6cosmosdb.documents.azure.com:443/)

Database ID *

imagecollection

Collection ID *

com682imagecollection

Document *

```

{
  "id": "Id x",
  "filePath": "Path x",
  "fileLocator": "FileLocator x",
  "fileName": "triggerFormDataValue(...) x",
  "userID": "triggerFormDataValue(...) x",
  "userName": "triggerFormDataValue(...) x"
}

```

Advanced parameters

Search

Show all

Clear all

Connected to CosmosDBCOM682. [Change connection](#)

Once this has been completed the Cosmos DB stage has been completed. Next, define the response for the logic app. This is a HTTP response object.

ins
Errors
Info

When an HTTP request is received

+

Create blob (V2)

+

Create or update document (V3)

+

Add an action

×

×

≡

All
Built-in
Shared
Custom

Request
Built-in
☆

Response

EasyVista Service Manager ☆

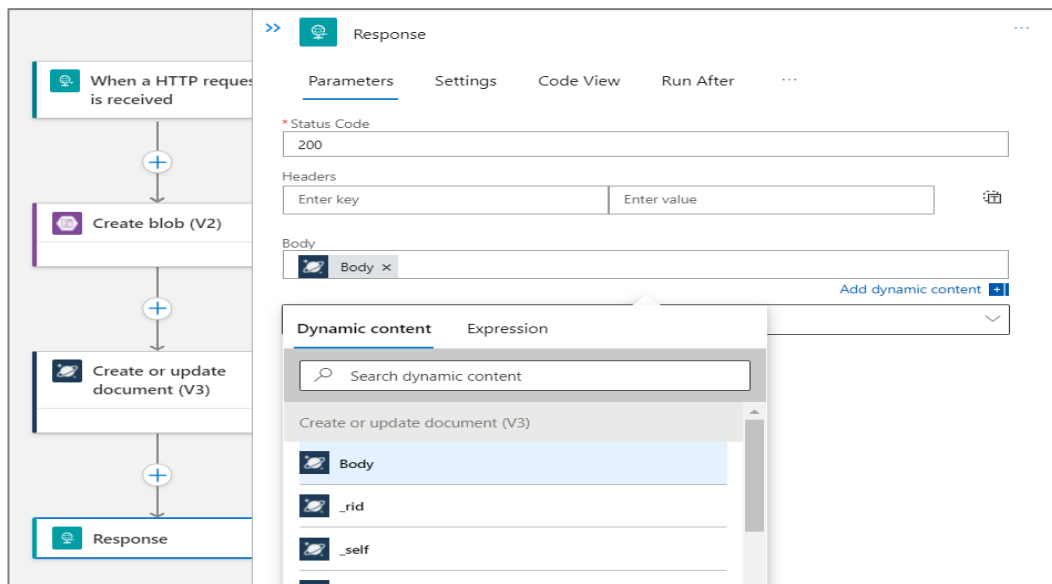
CreateResponseQuesti... Preview

UpdateResponseQuesti... Preview

ViewAllResponsesListQu... Preview

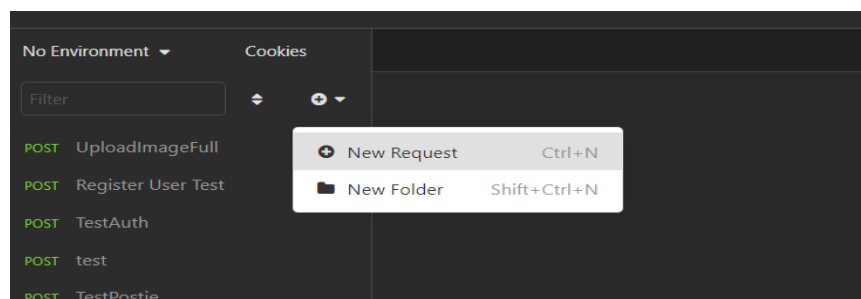
ViewListQuestionsWith... Preview

In this instance, we will return the body from the Cosmos DB operation.
Once done, save the logic app designer.



2.3.1.4. Testing the IUPS Endpoint

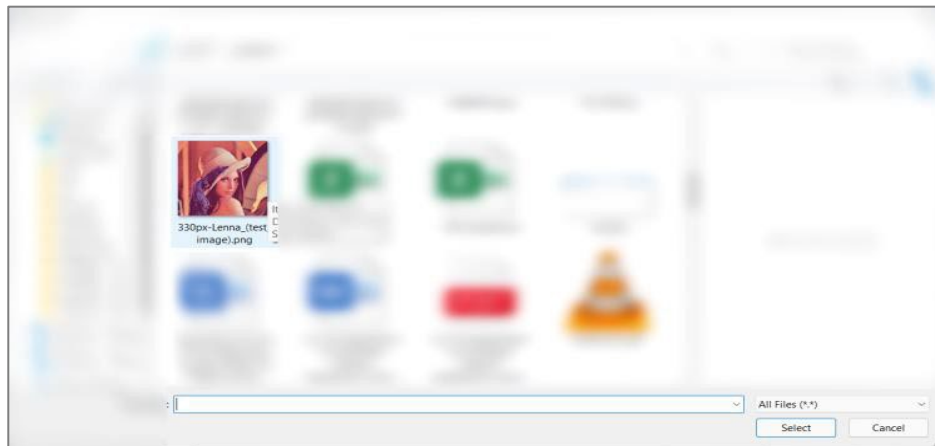
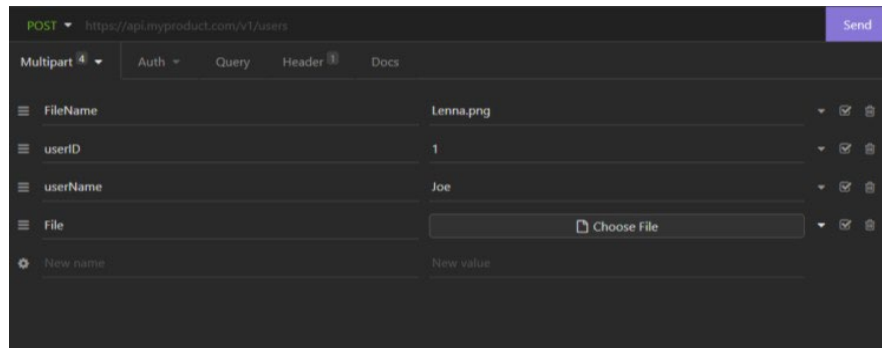
Create a new request in your API explorer tool, in the below we are using Postman/Insomnia.



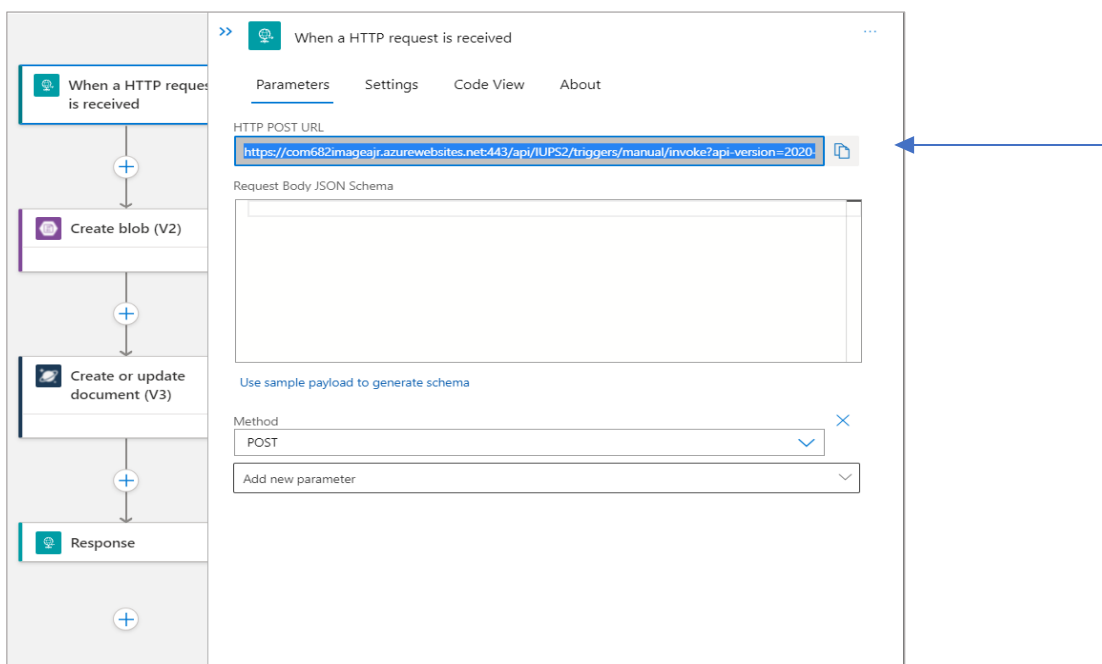
Set the request to POST

Provide 3 key value pairs for the initial text based form data. These should be Filename, userID and userName. We provided the test values Lenna.png (you can choose any PNG file from your local drive), 1 and Joe respectively. Following that, set a key called File and set its type to File.

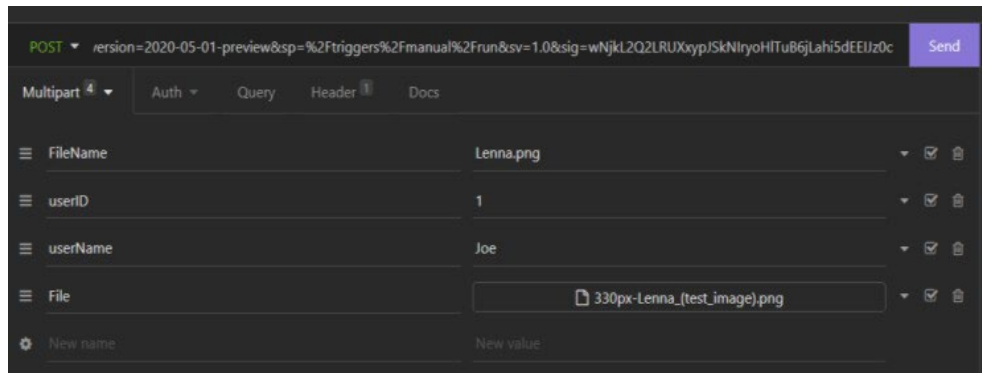
You will be presented a button to choose a file, click that and select a file to test upload.



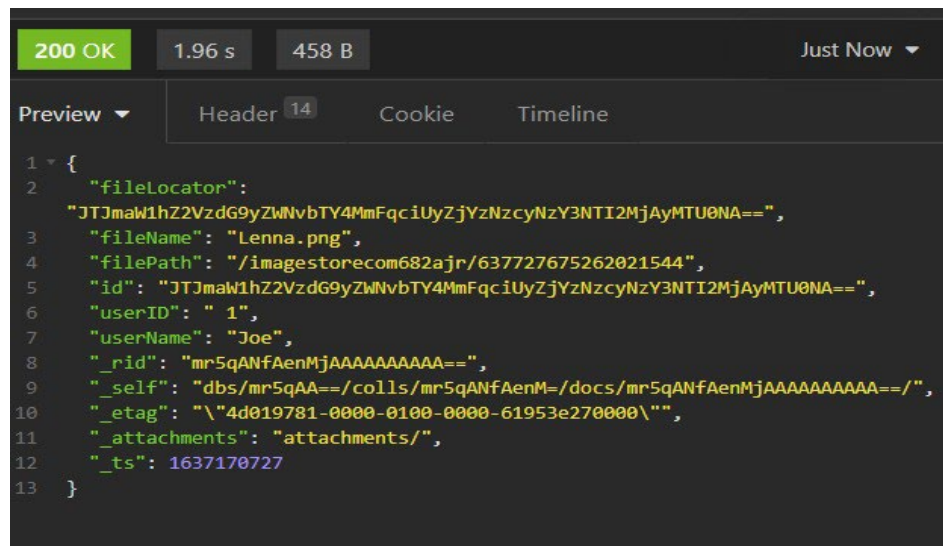
Note the IUPS URI and provide it to the testing tool.



Post the test data to the REST endpoint.

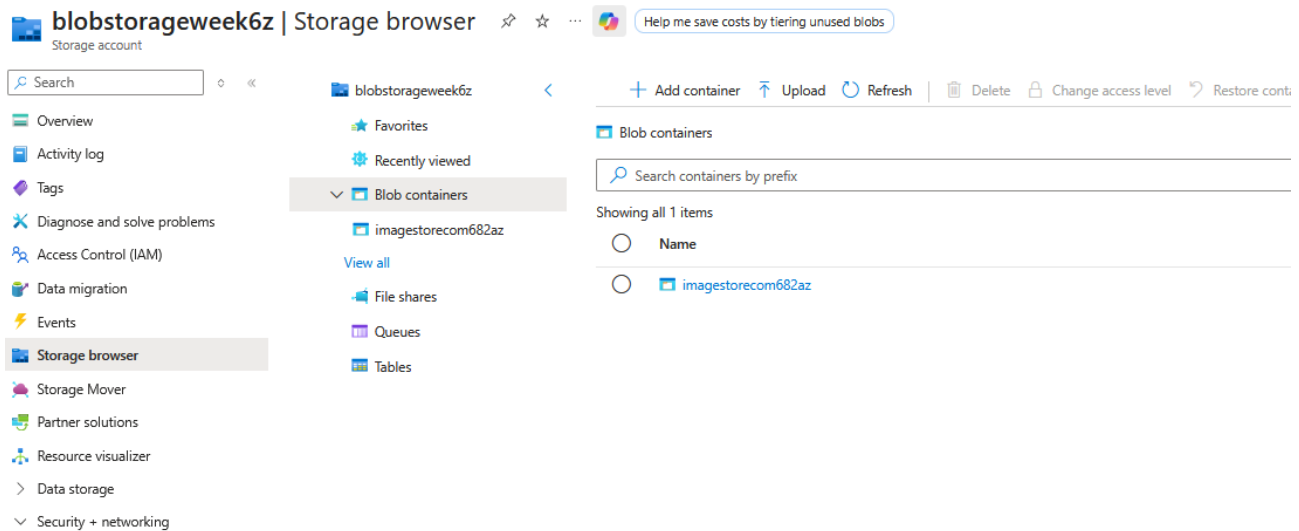


A response containing the Cosmos DB body will be provided if this was a success. This is shown below.

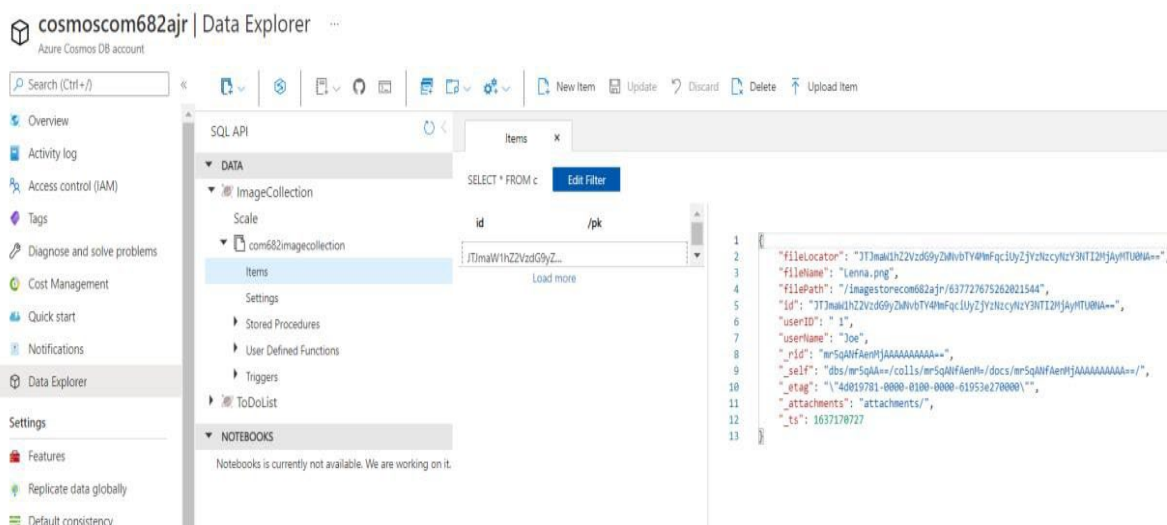


Next you can open the blob storage account to see if the file has indeed been stored, as you can see below, that was the case.

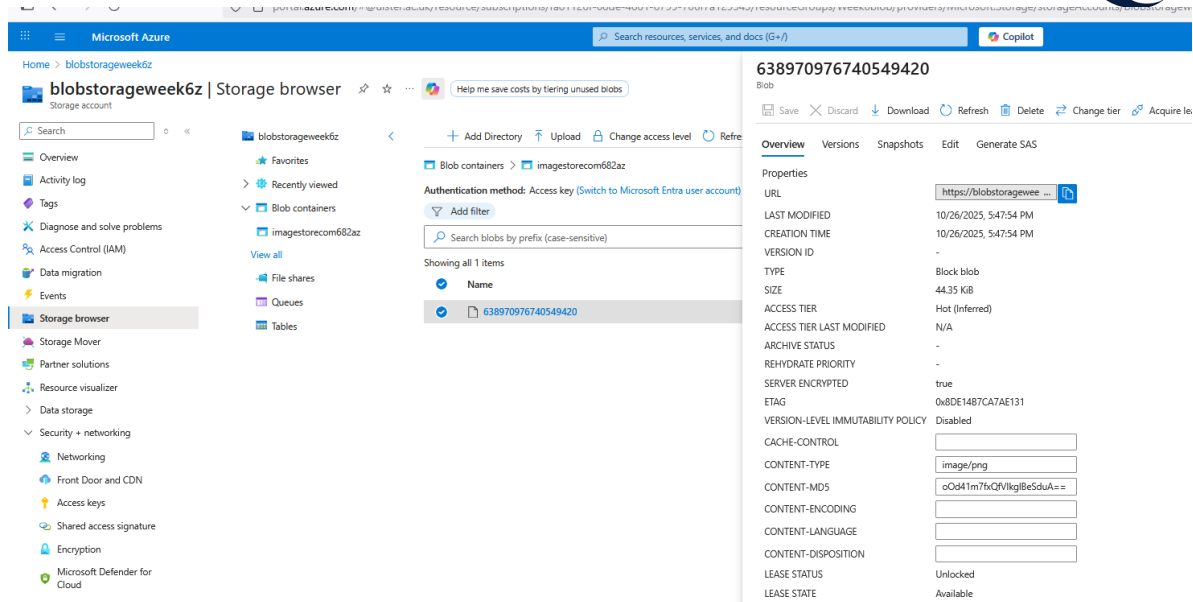
Home > blobstorageweek6z



Similarly, the storage of the document may be verified in the cosmos database data explorer.



Finally, you may view the image in a browser this may be achieved by obtaining the name of the storage account, in this case



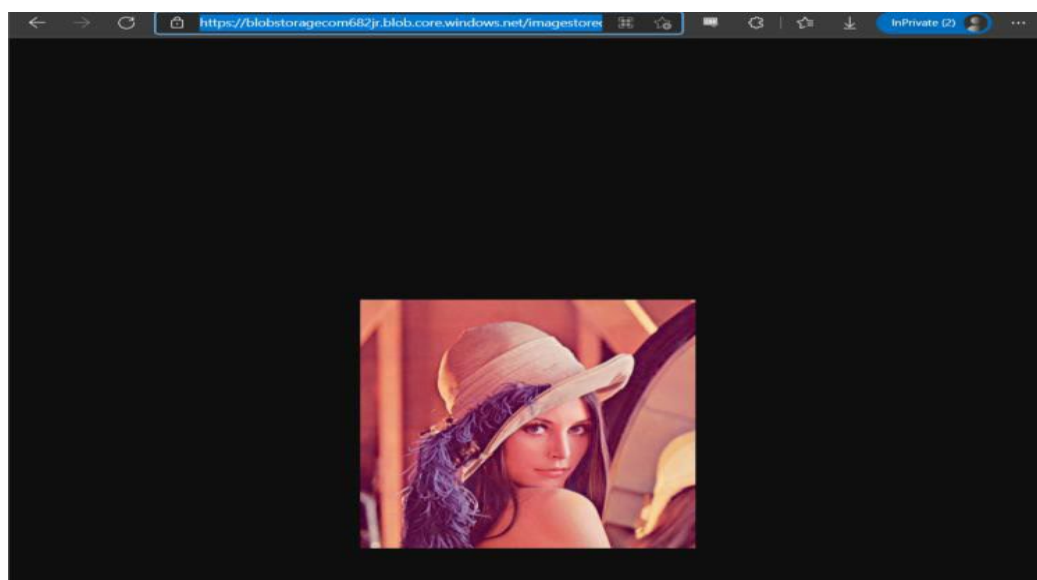
and concatenating that with the filePath record , in this case :

`/imagestorecom682ajr/637727602612267007 .`

To produce the following URI:

<https://blobstoragecom682jr.blob.core.windows.net/imagestorecom682ajr/637727602612267007>

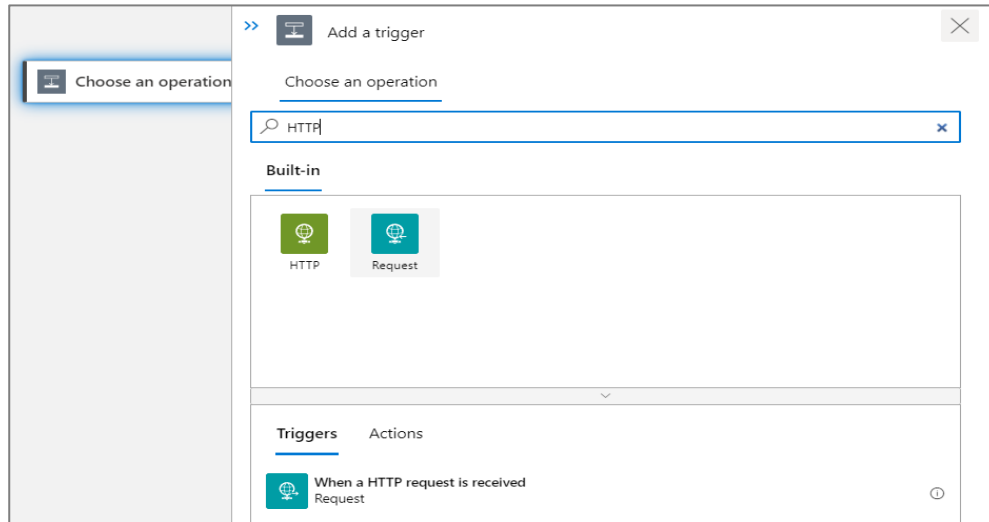
This URI may be viewed in a browser. If you cant view in the browser, its because the public access of resources are blocked at your storage account.



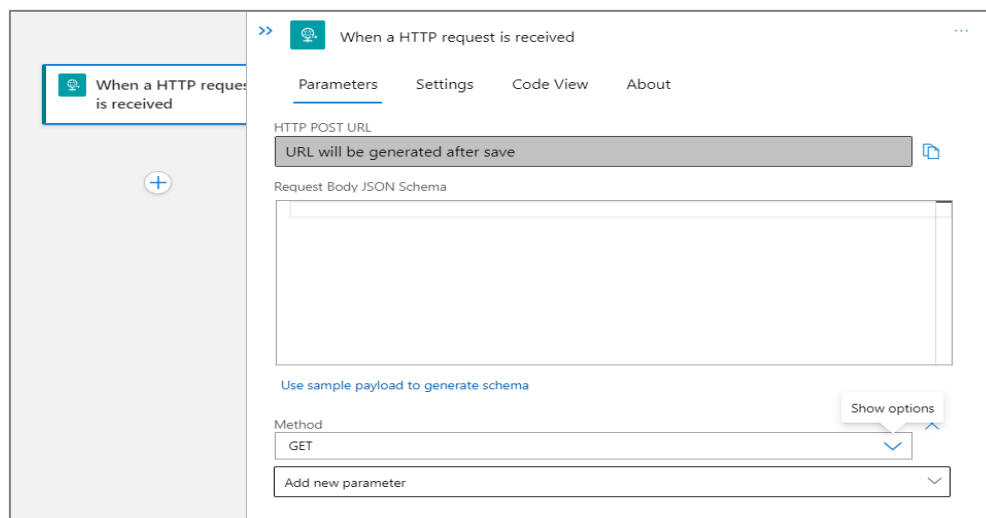
2.3.2. Establish a REST endpoint to provide a list of images stored

Next, create another endpoint to retrieve all image records from the CosmosDB. Call this **RAI** (retrieve all images).

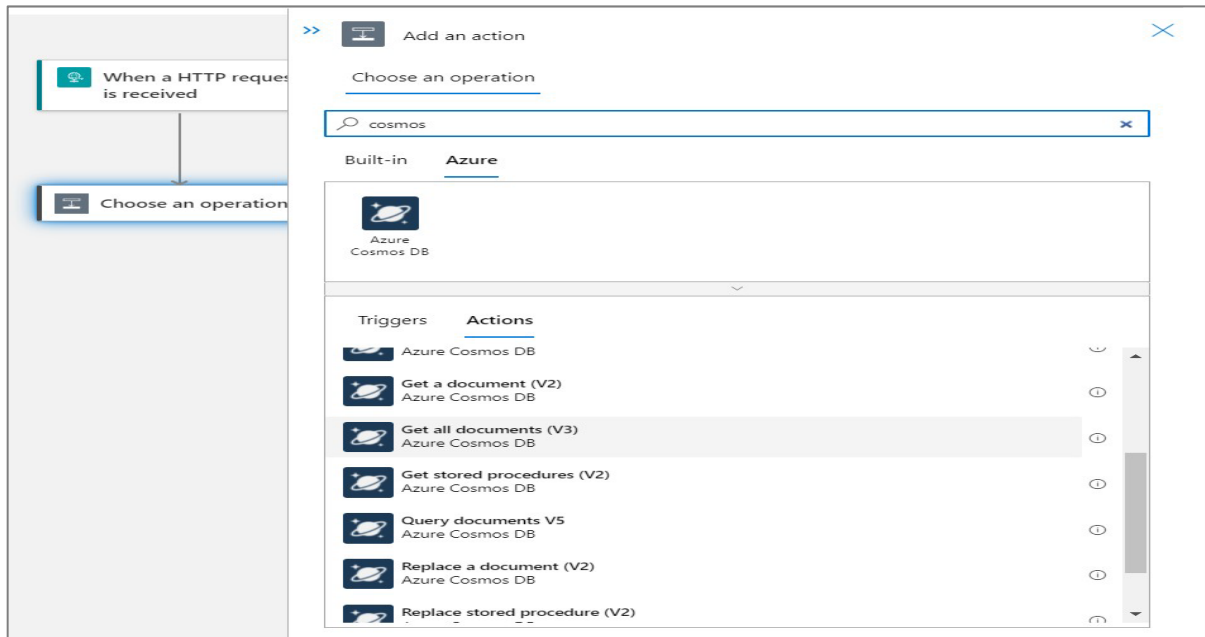
Create a HTTP request trigger.



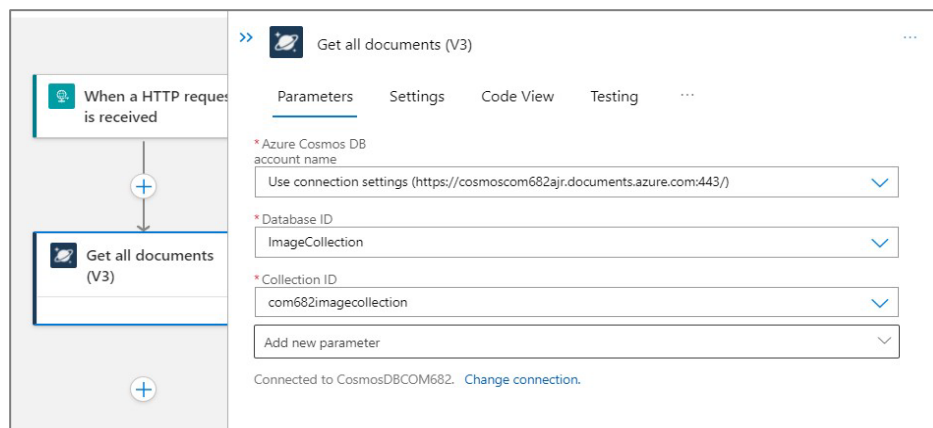
Set its method to GET.



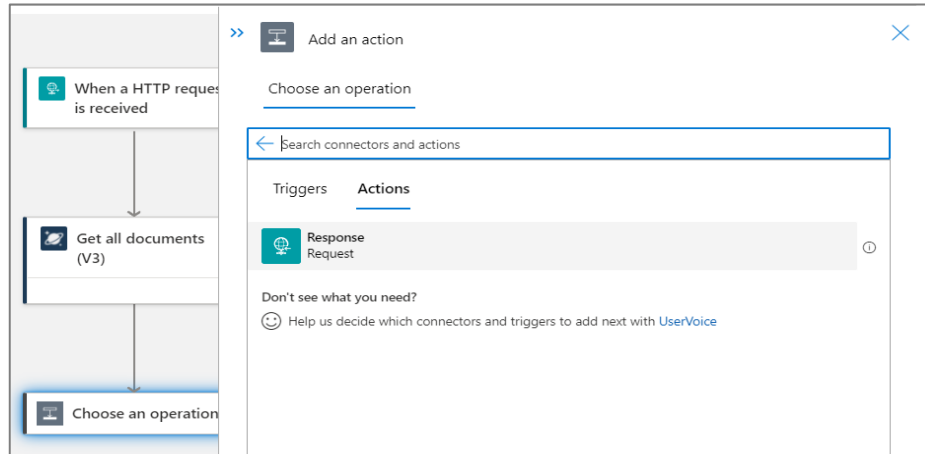
Add a stage incorporating the Azure function “Get all documents (v3)” which links to Cosmos DB.



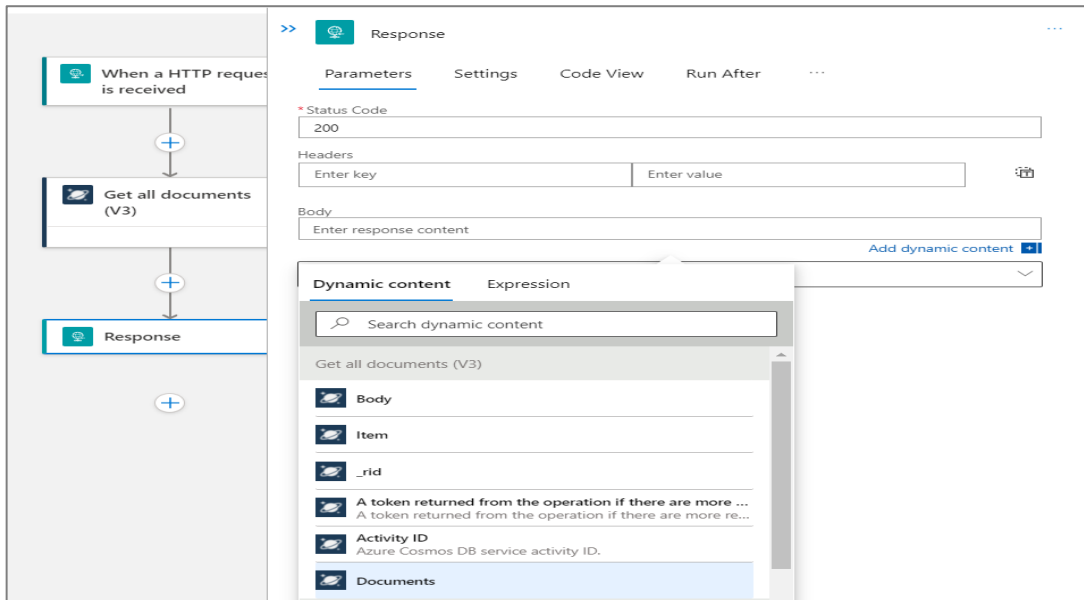
Configure the parameter's for the Cosmos DB connection appropriately. Ensure you specify the correct database and collection.



Add a HTTP response request block.



Set the body of the HTTP response block to “Documents”



Obtain the GET URL for the HTTP trigger and apply this to postman/insomnia or a browser to retrieve all records.

